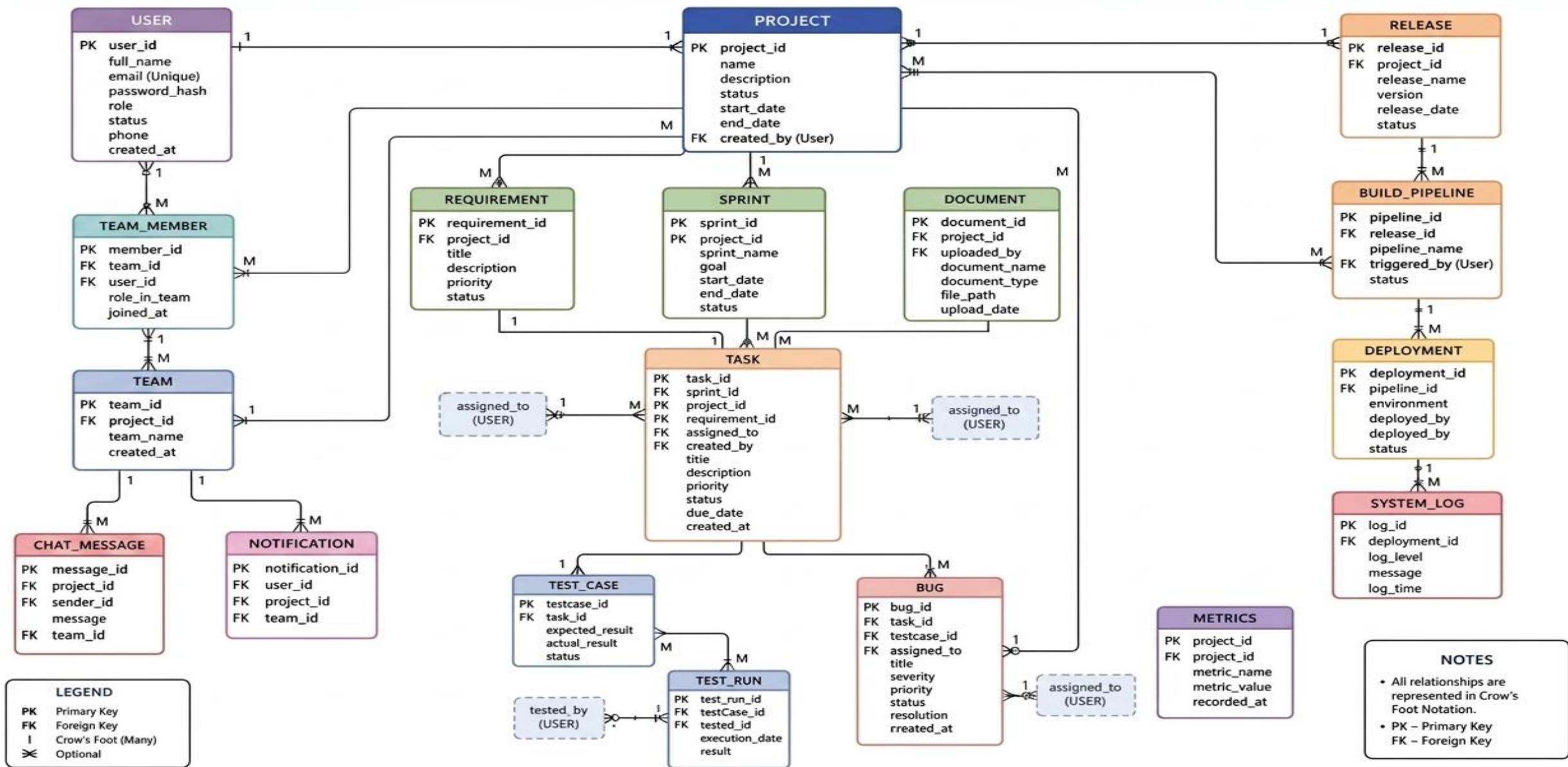


INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP

NeuroForce Enterprise SDLC Platform

Entity Relationship Diagram — Explanation

Entity Relationship Diagram (Crow's Foot Notation)



Why Choose Crow's Foot Notation Over Chen Notation?

- ✓ **Industry Standard** – Widely used for designing relational databases in real-world software projects.
- ✓ **Clear Relationship Representation** – Clearly displays **One-to-One**, **One-to-Many**, and **Many-to-Many** relationships.
- ✓ **Simple & Readable** – Easy to understand, especially for databases with many interconnected entities.
- ✓ **Implementation Friendly** – Closely matches relational databases such as **MySQL**, making implementation straightforward.
- ✓ **Ideal for NeuroForce** – Best suited for representing the complex relationships among **User, Project, Team, Sprint, Task, Bug, Release, Deployment, and Metrics**.

USER Entity

Purpose

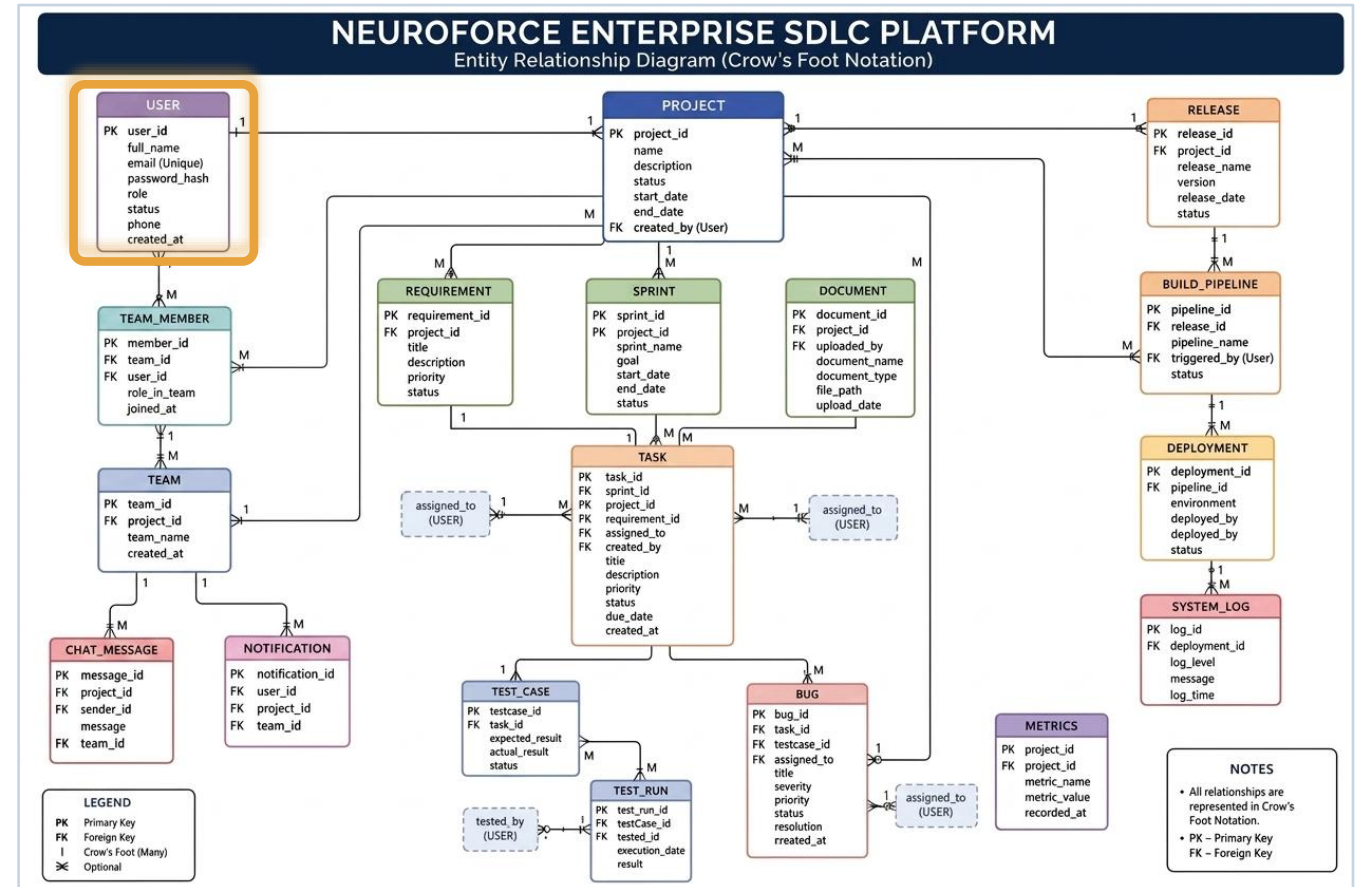
Stores every registered user of the platform — the single source of identity for the entire system.

Attributes

PK	user_id
	full_name
	email (unique)
	password_hash
	role
	status
	phone
	created_at

Key Points

- user_id is the Primary Key because it is guaranteed unique for every account.
- email is marked unique to prevent duplicate sign-ups and to support login.
- Connects outward to Project, TeamMember, Task, Bug, TestRun, Document, BuildPipeline, Deployment, ChatMessage, and Notification.



PROJECT Entity

Purpose

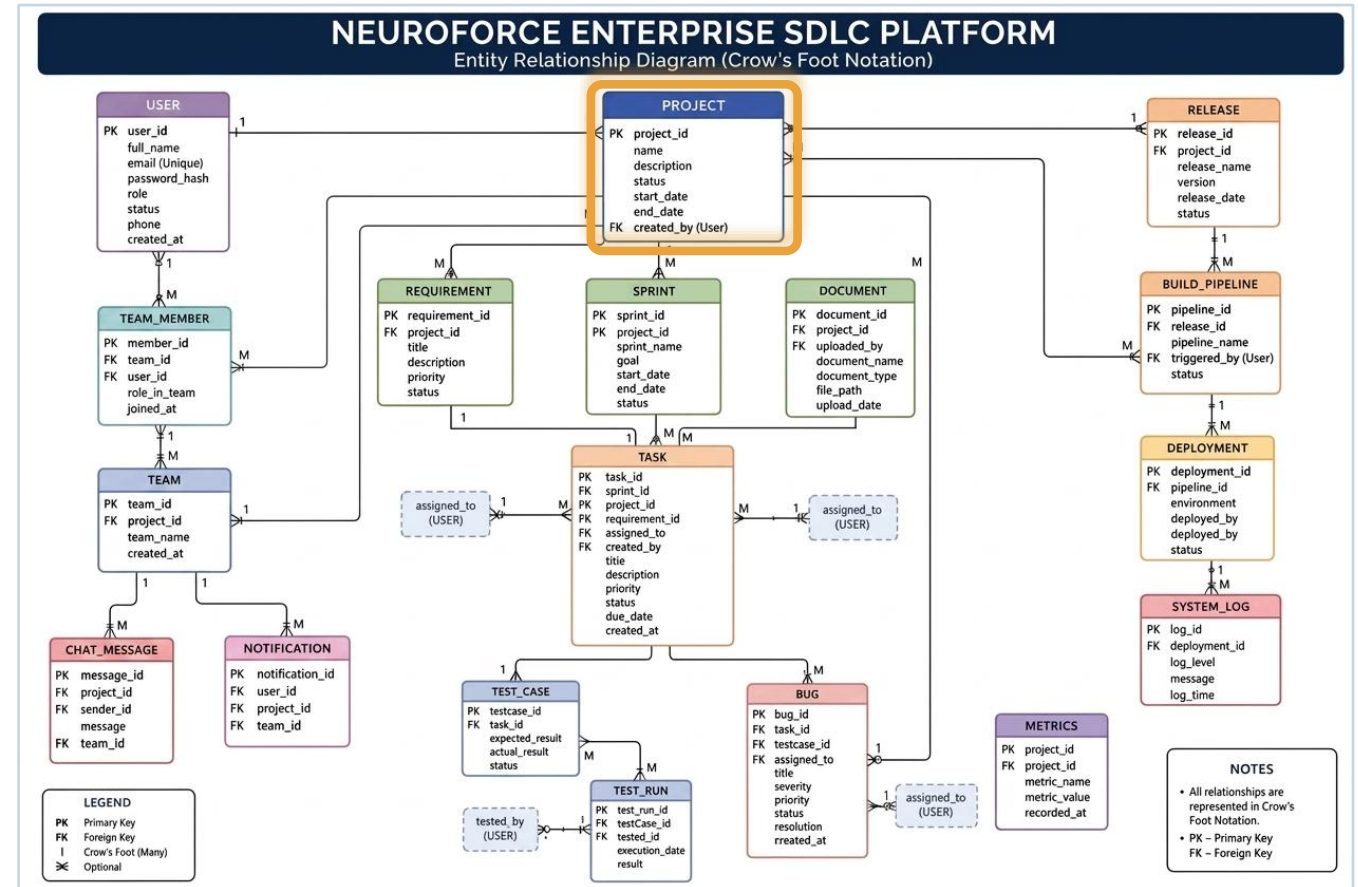
The central entity of the platform — every piece of SDLC work is organized under a Project.

Attributes

PK	project_id
	name
	description
	status
	start_date
	end_date
FK	created_by

Key Points

- created_by is a Foreign Key referencing USER — the project owner/creator.
- Directly parents Team, Requirement, Sprint, Task, Document, Release, Metrics, ChatMessage, and Notification.
- Every downstream entity ultimately traces back to a single Project.



REQUIREMENT Entity

Purpose

Captures the business or functional requirements that a project must fulfil, before any work begins.

Attributes

PK requirement_id

FK project_id

title

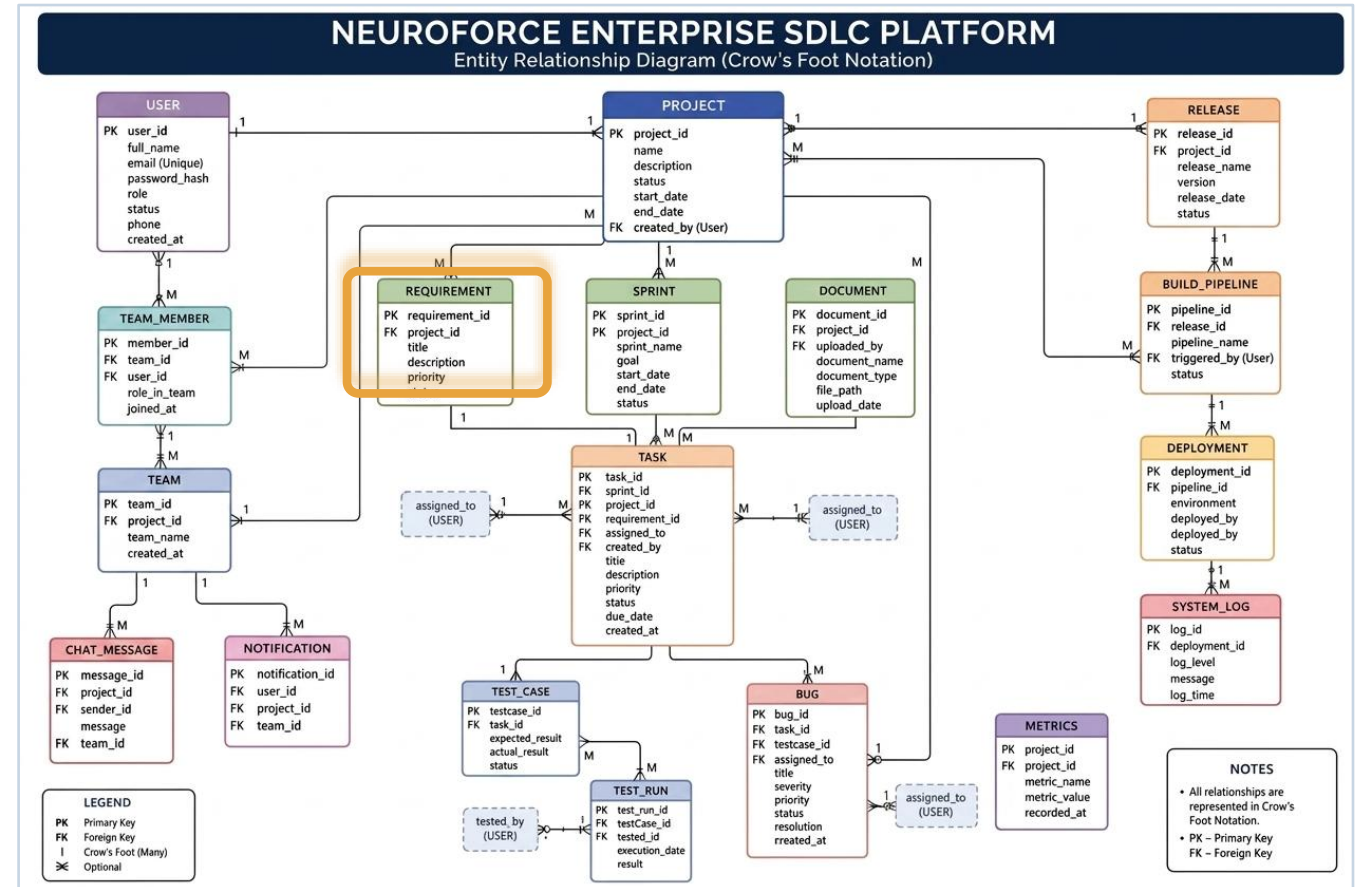
description

priority

status

Key Points

- Project → Requirement is one-to-many: one project can define many requirements.
- Requirement → Task is one-to-many: every task must trace back to a requirement.
- priority and status help teams decide what to build first and track completion.



SPRINT Entity

Purpose

Represents a fixed, time-boxed Agile iteration during which a defined set of tasks is completed.

Attributes

PK sprint_id

FK project_id

sprint_name

goal

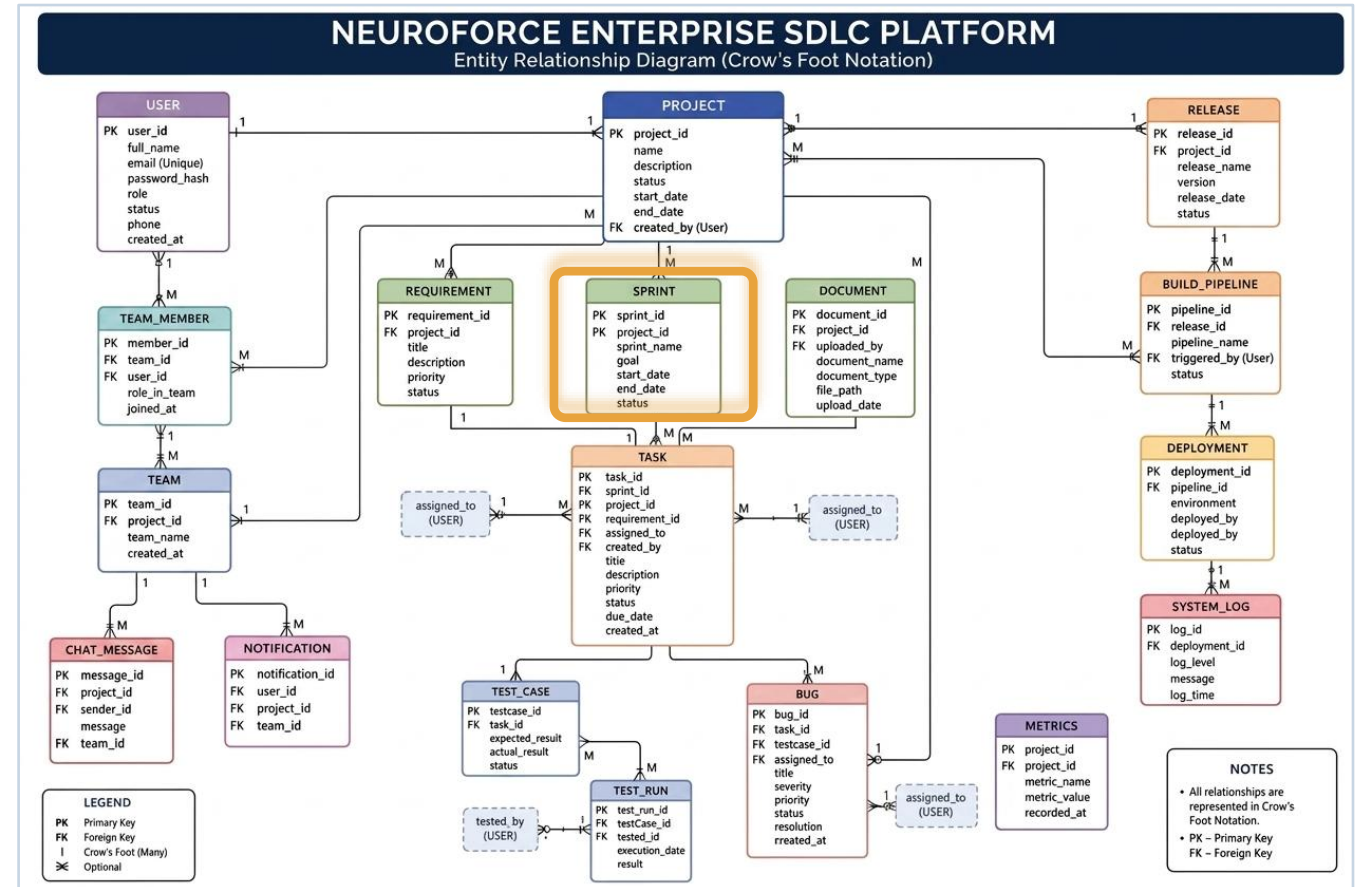
start_date

end_date

status

Key Points

- Project → Sprint is one-to-many: a project runs through many sprints over its lifetime.
- Sprint → Task is one-to-many: each sprint pulls in a batch of tasks to complete.
- goal, start_date, and end_date define the scope and time-box of the iteration.



TASK Entity

Purpose

The core unit of work — every requirement is broken down into tasks that get assigned, tracked, and completed.

Attributes

PK task_id

FK project_id

FK requirement_id

FK sprint_id

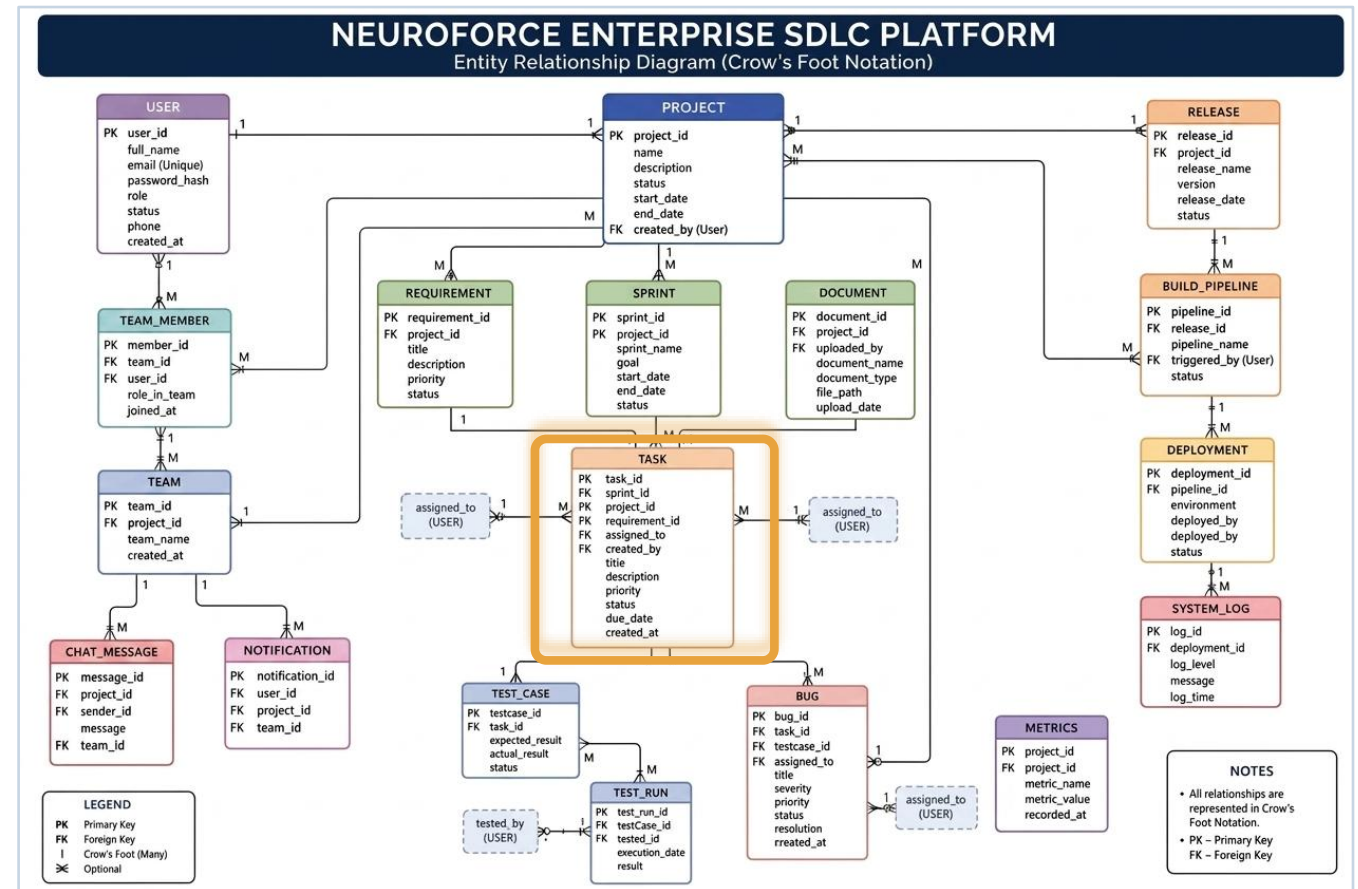
FK assigned_to

FK created_by

priority / status / due_date

Key Points

- Task lifecycle: created → assigned → in progress → completed, tracked via status.
- assigned_to and created_by both reference User, but represent different roles.
- Feeds directly into TestCase and Bug for quality tracking.



TEST_CASE and TEST_RUN

TEST_CASE

Defines what should be tested for a task and what result is expected.

PK testcase_id

FK task_id

expected_result

actual_result

status

- Task → TestCase is one-to-many: a task can have several test cases.

TEST_RUN

Records each execution attempt of a test case, and who ran it.

PK test_run_id

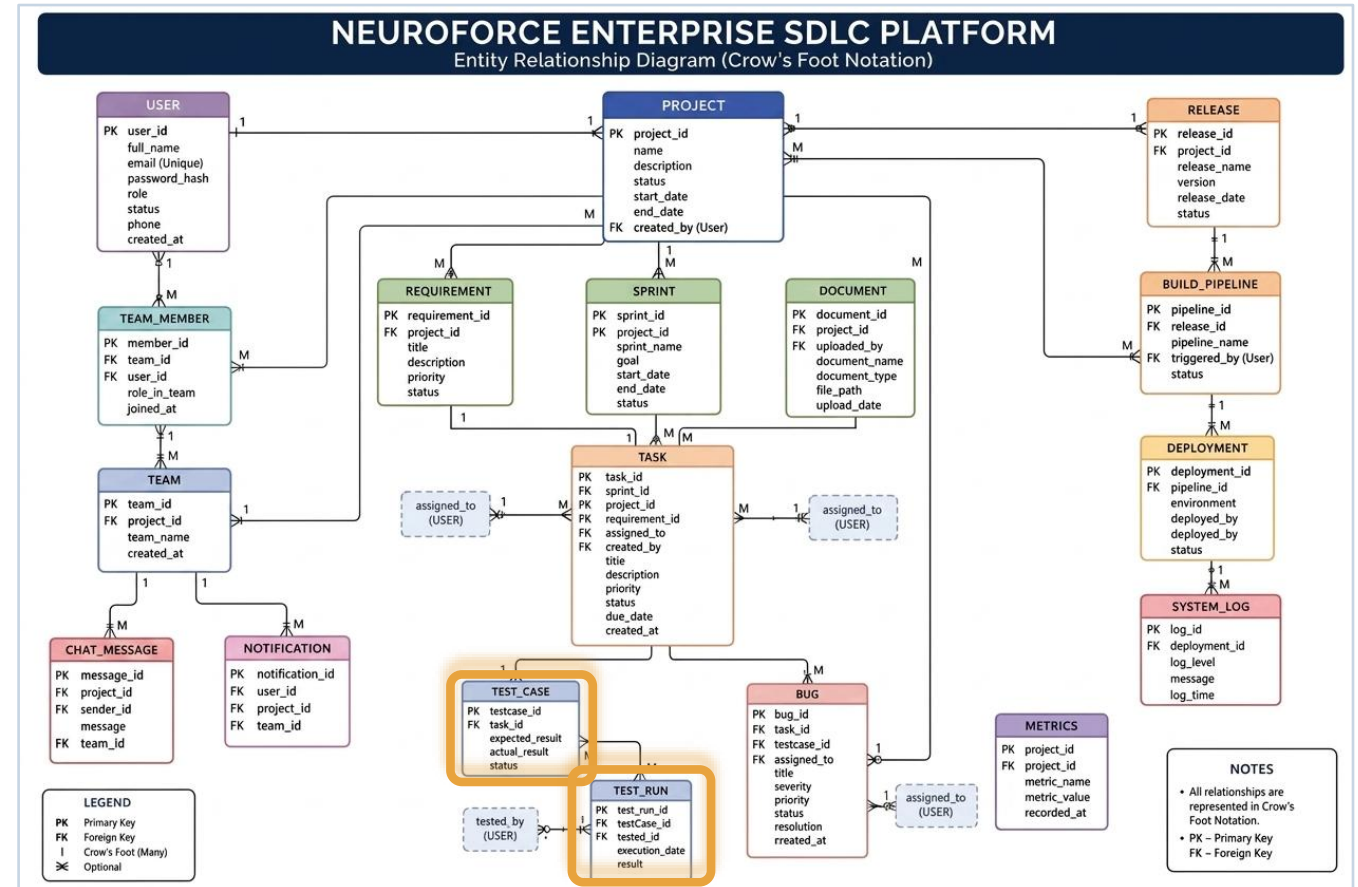
FK testcase_id

FK tested_by

execution_date

result

- TestCase → TestRun is one-to-many: one test case can be executed multiple times.



RELEASE, BUILD_PIPELINE & DEPLOYMENT

RELEASE

PK release_id

FK project_id

version / release_date / status

BUILD_PIPELINE

PK pipeline_id

FK release_id

FK triggered_by

pipeline_name / status

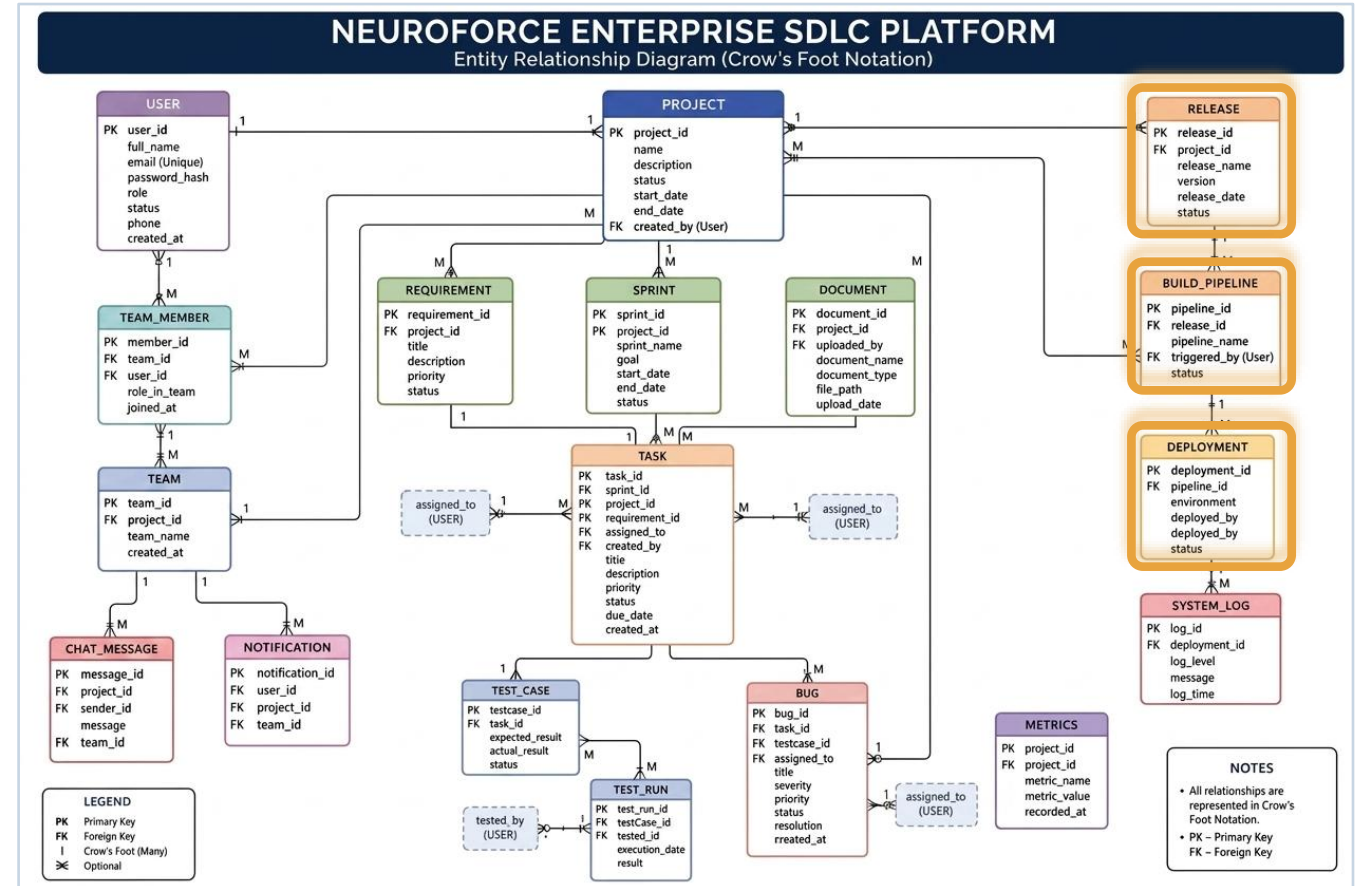
DEPLOYMENT

PK deployment_id

FK pipeline_id

environment / deployed_by / status

- Release → BuildPipeline → Deployment forms the complete CI/CD chain, each step one-to-many.



SYSTEM_LOG and METRICS

SYSTEM_LOG

Captures runtime logs, errors, and messages generated during a deployment.

PK log_id

FK deployment_id

log_level

message

log_time

- Deployment → SystemLog is one-to-many: each deployment can generate many log entries.

METRICS

Tracks project-level performance and quality metrics over time.

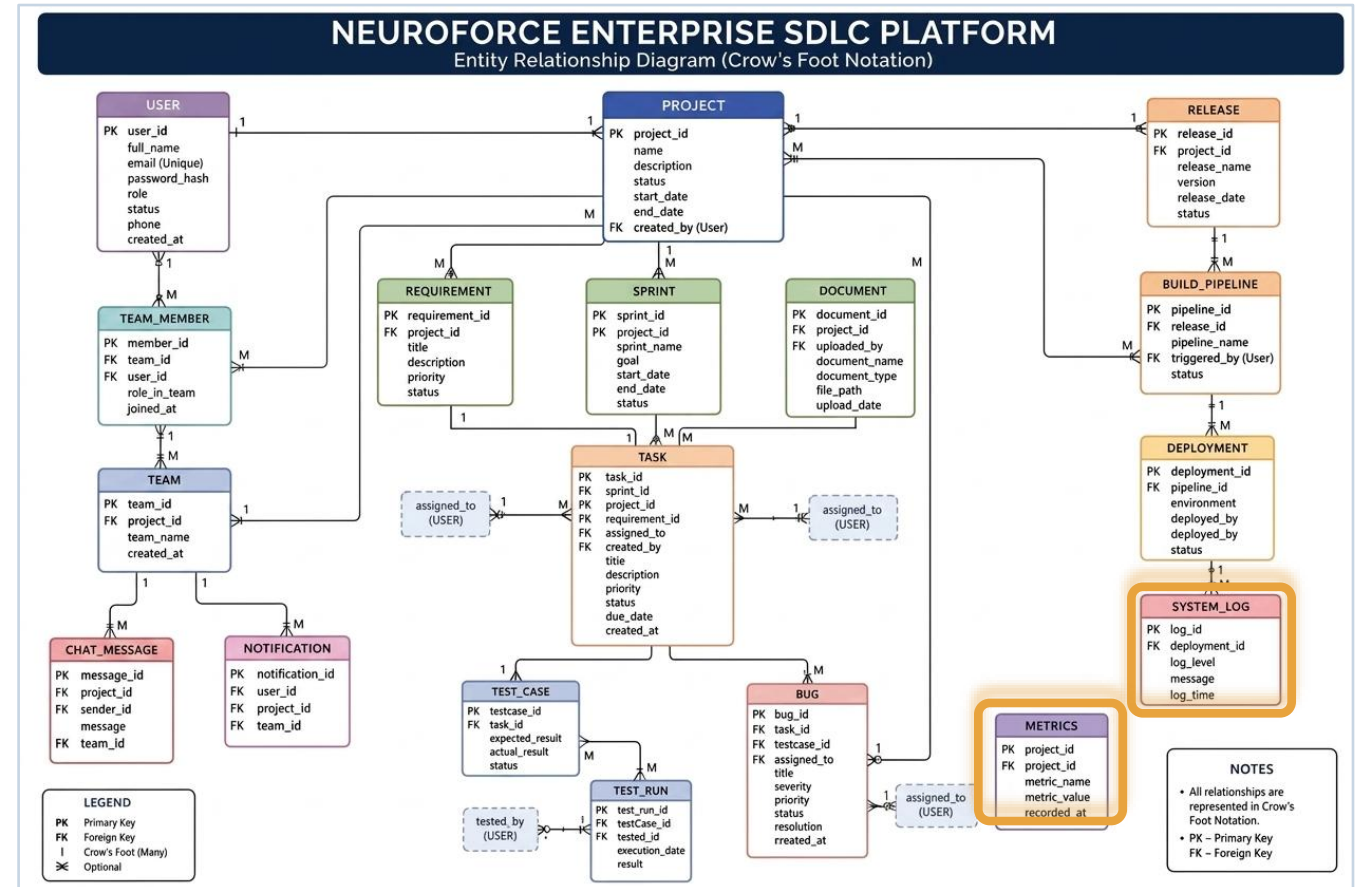
PK / FK project_id

metric_name

metric_value

recorded_at

- Project → Metrics is one-to-many: a project accumulates many recorded metrics.

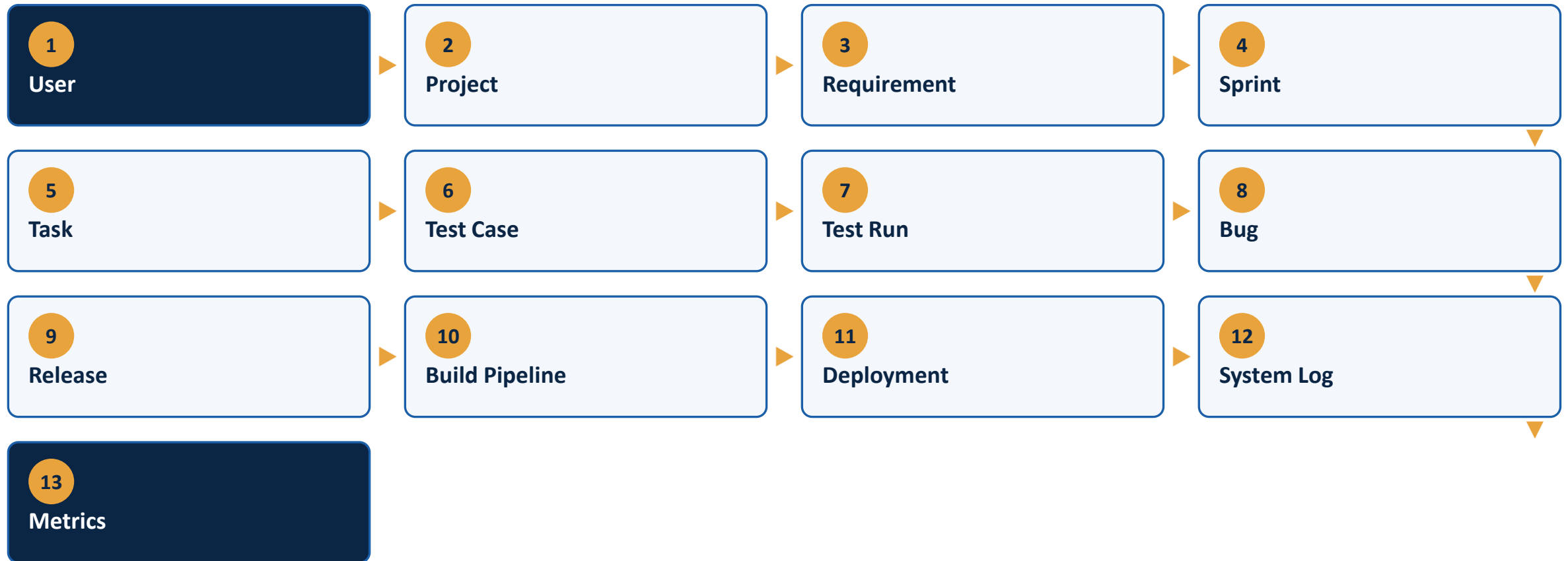




- User → Project
 - Team → TeamMember
 - Project → Sprint
 - Sprint → Task
 - TestCase → TestRun
 - Project → Document
 - Release → BuildPipeline
 - Deployment → SystemLog
 - Team → ChatMessage
 - Project → Team
 - Project → Requirement
 - Requirement → Task
 - Task → TestCase
 - Task → Bug
 - Project → Release
 - BuildPipeline → Deployment
 - Project → Metrics
 - Team → Notification

Crow's Foot reading: a single bar means “one”; a three-pronged crow's foot means “many.” Every relationship above reads left-to-right as One-to-Many — the parent has one row, the child table can have unlimited related rows.

Database Flow Through the SDLC



Data originates with a User and flows sequentially through project planning, execution, quality assurance, and delivery — finishing with Metrics, which feed insight back into the next planning cycle.